

App Name:

KD DD (*Karen & David: Designated Drivers*)

App Purpose:

Web app to drive business for a Designated Driver service - specifically, one called *Karen & David: Designated Drivers*

Technical Breakdown:

Database Schema Mapping

- database name: kd-dd.db
- collections:
 - Users - stores user/account information
 - Trips - stores booked trips
- *Users* collection fields:
 - `_id`
 - *Purpose*: the ID auto-assigned by MongoDB
 - *Example*: ObjectId("60c72b2f9b1d8b2bad754f21")
 - `userName`
 - *Purpose*: the user's user name for login information as string (most likely an email address)
 - *Example*: "abc@def.com"
 - `userPassword`
 - *Purpose*: the user's password for login information stored as hash number
 - *Example*: ???
 - `fullName`
 - *Purpose*: the name of the user as a string
 - *Example*: "David Watson"
 - `phoneNumber`
 - *Purpose*: the user primary phone contact as 10-digit number
 - *Example*: 1234567890
 - `emailAddress`
 - *Purpose*: the user primary email as string in validated email format
 - *Example*: "abc@def.com"
 - `creditCardNumber`
 - *Purpose*: the user's primary credit card payment method as 16-digit number
 - *Example*: 123456789012346

- creditCardName
 - *Purpose*: the name on the credit card as string (whether's the same as user or different)
 - *Example*: "David Watson"
- vehicleMake
 - *Purpose*: the make of the user's primary vehicle as string (probably validated against a list of known makes)
 - *Example*: "Honda"
- vehicleModel
 - *Purpose*: the model of the user's primary vehicle as string
 - *Example*: "Fit"
- *Trips* collection fields:
 - `_id`
 - *Purpose*: the ID auto-assigned by MongoDB
 - *Example*: ObjectId("60c72b2f9b1d8b2bad754f21")
 - `userID`
 - *Purpose*: the record ID of the trip user
 - *Example*: ObjectId("60c72b2f9b1d8b2bad754f21")
 - `startAddress`
 - *Purpose*: the full starting or pick up address as address object
 - *Example*: { streetNumber, streetName, unitNumber, cityName, countyName, provinceName, postalCode }
 - `endAddress`
 - *Purpose*: the full ending or drop off address as an address object
 - *Example*: { streetNumber, streetName, unitNumber, cityName, countyName, provinceName, postalCode }
 - `totalDistance`
 - *Purpose*: the total distance of the trip in km as a number
 - *Example*: 42
 - `extraCalculations`
 - *Purpose*: list of optional aspects to the trip, such as stopovers or wait time, as an object
 - *Example*: { stopOvers { total, [{ address object, waitTime }] }, startWaitTime }
 - `totalCost`
 - *Purpose*: the total calculated cost of the trip as a number (representing CAD)
 - *Example*: 98.15

API Endpoints

- GET routes

- '/'
 - *Returns:* main site page with Hero section, header, and footer
- '/trip-calculator'
 - *Returns:* fare estimator page
- '/account-create'
 - *Returns:* page to create an account
- '/account-login'
 - *Returns:* login page for an account
- '/account-info'
 - *Returns:* profile page for an account, showing profile info (taken from DB)
- '/account-trip-history'
 - *Returns:* page showing trip history for logged in user; if no user is logged in, redirects to '/account-login'
- POST routes
 - '/trip-calculator'
 - *Data Sent:* details from the fare estimator page - pickup address, dropoff address
 - *Data Returned:* trip details
 - '/account-create'
 - *Data Sent:* details needed to create the account
 - *Data Returned:* redirect to login page - GET '/account-login'
 - '/account-info'
 - *Data Sent:* changes to the account info to update it
 - *Data Returned:* new account info or "refresh" of the GET '/account-info' route

Features List:

Base (for MVP)

- *feature:* users can choose their start and end locations and system will calculate the estimated fare - based on the business fare calculations - and display that to the user
- *feature:* allow user to book a trip based on the fare estimate and get from user the following information - name, phone number, email, credit card number, name on credit card, vehicle make, vehicle model, vehicle colour, name of owner of vehicle
- *feature:* (basic) display fare costs or calculation breakdown
- *feature:* when users go to book trip, show calendar of availability and let users pick time
- *feature:* allow users to book trip for "ASAP", meaning pickup time is immediate or as soon as possible

- *feature*: have ability for users to create an account either directly or after booking a trip
- *feature*: have ability for users to log into account
- *feature*: if users create account after trip booking, have trip information be used to create account
- *feature*: if users create account directly, give them two options:
 - provide email and password to create account, followed by 2FA using email
 - provide SSO account creation using Google (+ FB, Apple, etc.?)
- *feature*: create profile page for logged in users that shows account info but also allows for updating of account info
- *feature*: track and save booked trip info for users with account and who are:
 - logged in when booking
 - book first but then log in / create account afterward
- *feature*: allow users to see their booked trip history with following information:
 - pickup address
 - dropoff address
 - any wait time
 - any extra stopovers
 - total trip distance (in km)
 - subtotal trip cost (pre-tax)
 - tax (if business gets to point where tax has to be charged)
 - if there's no tax, show 0
 - total trip cost (subtotal + tax)
 - any tip given (outside of total trip cost)
 - total paid (total trip cost + tip given)
 - payment type
 - if payment gets spit between different types, show all types and amount paid with each

Future (for value-add)

- *future feature*: allow users to have multiple cards and vehicles attached to their single profile
- *future feature*: have some way to show a time estimate for users who book ASAP
- *future feature*: have time estimate be a clock that counts down rather than just static
- *future feature*: show vehicle location when on the way to pickup